

Harness architecture, test status, and replaceable custom code

2026-08-12

Reviewed 2026-08-12 against `feat/decode-provenance-2026-08-08`. Line citations are to that commit.

Testsuite

It works. `python3 -m pytest harness/ -q` → **446 passed in 77s**, zero collection errors. Stdlib-only: no Java, no network, no GPU, no secrets. CI (`.github/workflows/ci.yml`) runs three jobs on every push and PR — the suite, corpus export/audit agreement, and a frozen-artifact JSON/JSONL parse check — with java deliberately off `$PATH` so any test that starts shelling out to a real verifier fails loudly.

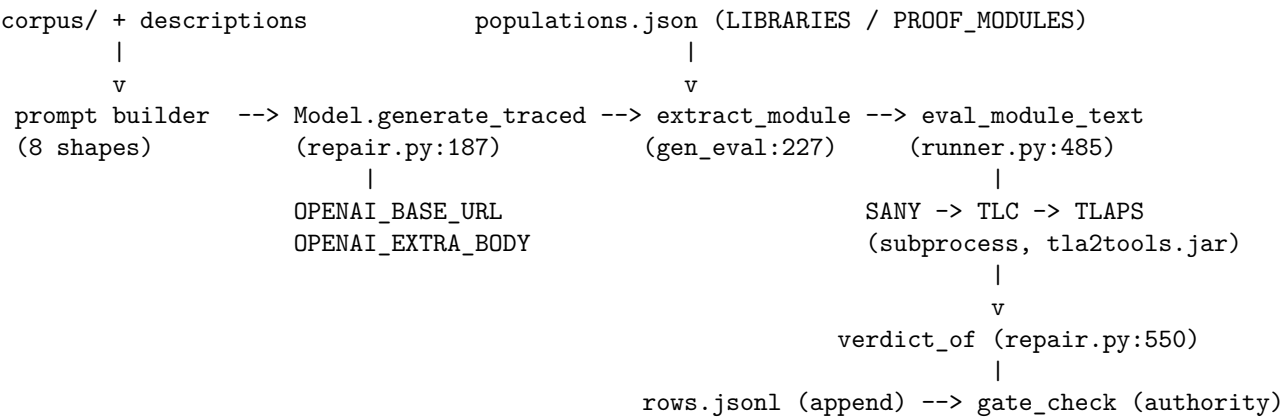
What it does not cover:

gap	size
modules with no test file	13 of 33 (~2,400 of ~9,600 lines, 25%)
<code>repair.py</code> — no <code>test_repair.py</code>	881 lines, second-largest module
<code>__main__.py</code> — untested	the entire subcommand→function dispatch
all of <code>tools/</code>	includes the 3 files that produce published statistics
pytest config / <code>conftest.py</code>	neither exists

`repair.py` is the one that matters: it holds **both** the model layer (`Model`, `OpenAICompatModel`, `make_model`) **and** `verdict_of (:550)`, the pass criterion every reported number depends on. It is exercised indirectly through `test_decoding.py` and `test_gen_eval.py`, but the escalation ladder (`repair_spec :629`), `localize :355`, and `run_repair :808` have no direct coverage.

`tools/rowlevel_power.py`, `tools/passk_curve.py` and `tools/w4_difficulty_report.py` are untested and contain ~190 lines of hand-written statistics, including a modified Lentz continued-fraction `betainc`. The harness has tested equivalents of three of those functions; the `tools/` copies are separate code.

Pipeline



Entry points are 8 subcommands in `__main__.py`: `run`, `repair`, `gen-eval`, `gate-check`, `replay`, `semaudit`, `gate1-report`, `proof-traces`.

runner.py (588 lines) is the verification core: check_sany :171, check_tlc :255, check_tlapm :271, check_apalache :298, all funnelled through run_cmd :127. Only tools/tla2tools.jar is on the classpath; CommunityModules-deps.jar is deliberately excluded (compiled against a newer tla2tools, breaks TLC) and community modules are supplied as plain .tla on the library path. Toolchain paths are hardcoded relative to the repo root — there is no TLA_TOOLS_JAR env var.

gate_check.py is the single scoring authority: it recomputes pass@k from rows.jsonl with keep-first dedup and hard-fails a run above 5% api_error.

Routing

Eight distinct prompt shapes exist. The one that matters for train/eval agreement:

shape	built by	used by
P1 framing-A generation	gen_eval.build_generation_prompt :394	Gate-2 framing A, replay — nothing else
P2 framing-B repair	gen_eval.build_repair_prompt :421	Gate-2 framing B, repair_harvest, and the repair SFT renderer
P4 W2 generation	w2_loop.generation_prompt :138	corpus generation, difficulty probe, SFT prompt_style="generation"
P6 bare NL	corpus_prep.to_harmony_sft :385 (default)	every SFT corpus through W4-diamond-gold

P1 has never been an SFT target. build_generation_prompt is imported by exactly two modules, gen_eval.py and replay.py; corpus_prep.py never references it, and PROMPT_STYLES is ("bare", "generation") with no style that renders P1. So framing-A evaluation has always used an output contract (bare module, cfg-derived identifier block) that no training corpus ever demonstrated. P2 is the exception: the repair SFT renderer calls build_repair_prompt directly, which is why the mech corpus matches its eval shape.

Six routing hazards

1. **OPENAI_EXTRA_BODY can override model itself.** repair.py:245 merges it after the body is built, so extra-body wins by design. A stale value carrying "model" changes what is served while the ledger still records the intended id (gen_eval.py:617). decode_params_sha256 hashes the merged body and would expose it — but see hazard 4.
2. **Nothing validates that the endpoint serves the requested model.** tools/smoke/serve_pre-flight.py:49 does exactly this check, and no harness module calls it. An OpenAI-compatible server that ignores an unknown model field answers from whatever it has loaded.
3. **gen-eval does not record the endpoint.** Its config.json (:742) captures run id, model id, corpus, k and holdout hash — but not OPENAI_BASE_URL. repair.run_repair (:837) does capture it. The two main drivers differ.
4. **Decode provenance is on 2 of 9 model-calling paths, and 0 of 172 ledgers.** generate_traced_compat is called only from gen_eval.py:59 and replay.py:147; repair.py, w2_loop.py, repair_harvest.py, proof_gen.py and w4_scenarios.py call plain generate with no seed and no provenance. Verified directly: of 172 rows.jsonl files under results/runs/, **zero** contain decode_seed or extract_divergent. The feature shipped in 21f664b6/00d6c2dd has not yet been exercised by any run.
5. **replay never checks the backend it replays against.** It rebuilds the model as make_model(f"openai:{row['model']}") (:144) and takes the endpoint from current env. Rows carry backend_sha256; replay does not compare it, so replaying against a different serve reports DIVERGENT with no indication the backend moved. It does check prompt drift (:127).
6. **Two different stub classes share one id.** --model local-stub routes to gen_eval.LocalStubModel (:872, no generate_traced, empty provenance); --model stub routes to repair.LocalStub (:103, native tracing). Both report id = "local-stub-v1", so the ledger cannot distinguish them.

Writer locking is uneven

The 2026-07-15 duplicate-row incident produced a PID lockfile — on one path only.

driver	ledger mode	lock
<code>gen_eval.run_gen_eval</code>	append	<code>writer.lock</code> , <code>O_EXCL + PID + atexit :723</code>
<code>repair.run_repair</code>	"w"	none; refuses if <code>rows.jsonl</code> exists
<code>w4_difficulty.run_probe</code>	append	none (in-process <code>threading.Lock</code> only)
<code>w2_loop</code> , <code>w4_scenarios</code> , <code>repair_harvest</code> , <code>w27_scaffold</code> , <code>proof_gen</code>	own JSONL	none

`gate_check`'s keep-first dedup is the cure rather than the prevention. Of ten JSONL readers across the tree, only `w4_difficulty.load_done` survives a truncated tail line; three different dedup keys are in use.

Frameworks that could replace custom code

1. vLLM guided decoding — the grammar is already written, and this needs no code change.

`harness/grammars/tla_module_v0.ebnf` is authored in the **xgrammar EBNF dialect**, which is **exactly what vLLM's guided_grammar consumes**, and `tools/smoke/grammar_check.py` already validates it compiles under xgrammar. The serve runs vLLM 0.22. Nothing in the harness passes it: `guided_grammar` appears in `grammar_check.py` and nowhere else. Because `repair.py:245` merges `OPENAI_EXTRA_BODY` into the request body verbatim, this can be tested by setting an env var — no harness edit at all. Given 69–90% of failed draws are parse failures, this is the cheapest untried lever in the project.

2. Statistics → scipy. ~190 lines across three `tools/` files, all untested, including `_betacf/betainc/_beta_ppf` (a continued-fraction incomplete beta plus bisection inverse) that `scipy.special` provides, and Wilson intervals that `statsmodels` provides. The stdlib-only policy is deliberate (`requirements-dev.txt:3-8`) and is what keeps CI dependency-free; adopting `scipy` in `tools/` only would preserve that for `harness/`.

3. Model client → litellm or the openai SDK. Would replace 258 lines (`repair.py:61-318`) and two divergent retry loops — `AnthropicModel` retries 3× on {429,529,500}, `OpenAICompatModel` 6× on eight codes, both **linear** backoff, neither reads `Retry-After`. Caveat: several parts are not stock — reasoning-channel fallback across three field names (:283), per-choice seed echo (:275), the extra-body merge, and a custom User-Agent because Cloudflare 403s `urllib`'s default (:256). A migration keeps those as wrappers.

4. Locking → filelock. Replaces the 15-line PID lockfile and, more usefully, makes it cheap to extend locking to the five unlocked writers.

5. Near-duplicate detection → datasketch. `harness/corpora.py` (~100 lines) is a hand-rolled blake2b-shingle MinHash/Jaccard. It is the contamination gate, so it is load-bearing and currently untested at scale.

6. Row schema → pydantic or jsonschema. There is no row-schema validation anywhere; the only mechanism is a field-name tuple (`w4_difficulty.py:269`). Every other writer emits free-form dicts and missing keys silently become `None`. Several past defects were of exactly this shape.

7. tlakit (LUC-AI4FM's own TLA+ Python client) overlaps `runner.py`'s subprocess layer. Not evaluated here.

Do not replace

- **`runner.run_cmd :127`.** It is not a reimplement of `subprocess.run(timeout=)`: it sets `start_new_session=True` and `os.killpg` on the **normal-exit** path as well as on timeout, because `tlapm` returns success while `z3` and `Isabelle` workers survive. Both kills carry dated incident comments (leaked `polym1` at 25% RAM each; forty orphaned `z3`s reparented to pid 1). Every verifier call depends on it.
- **The two frozen extractors.** `gen_eval.extract_module` takes the *first* match and `repair.extract_candidate` the *last*, with different regexes. `decoding.py:102` measures the divergence rather than fixing it, deliberately: unifying them would retroactively invalidate every ledger in `results/runs/`.

Suggested order

1. Wire `guided_grammar` through `OPENAI_EXTRA_BODY` on the next serve. No code change; directly targets the dominant failure mode.
2. Call `serve_preflight` from `gen_eval` startup, and record `OPENAI_BASE_URL` in `config.json`. Closes hazards 2 and 3.
3. Route `generate_traced_compat` on the remaining seven model paths, so provenance starts appearing in ledgers.
4. Give `repair.py` a test file, starting with `verdict_of` and its population branches.
5. Collapse the two stubs to one class, or give them distinct ids.